Brain Doc

The Rules

• Starting capital: $1,000,000 (fake money on Roostoo's mock exchange)
• Asset: BTC_USDT (Bitcoin priced in USDT stablecoin)
• Spot trading only — we buy BTC, we own it. Price moves affect us directly.
• No shorting — we can ONLY profit when price goes UP. When price is dropping, we sit in cash (USDT).
• No leverage — we trade only with what we have. Cannot borrow money to trade bigger.
• No market-making — cannot place buy AND sell orders simultaneously to profit from the gap.
• No arbitrage — cannot exploit price differences across different markets/coins.
• Directional trading only — we predict which direction the price will move and bet on that.
• API rate limit: 1 trade per minute maximum.
• Must be open-source on GitHub.

The Fees

Every time we trade, Roostoo takes a small cut:
• Market order (taker): 0.1% fee. We say 'buy now at current price.' Fast but expensive.
• Limit order (maker): 0.05% fee. We say 'buy only at this specific price.' Slower but HALF the cost.
WE ALWAYS USE LIMIT ORDERS. Over hundreds of trades on $1M, the fee difference saves us thousands of dollars.
Round-trip cost (buy + sell): 0.05% + 0.05% = 0.10% total. This means every trade must make MORE than 0.10% profit just to break even.

Main Formula

0.4 x Sortino + 0.3 x Sharpe + 0.3 x Calmar

• Sortino ratio (40% weight): Measures returns relative to DOWNSIDE volatility only. If your bot **makes money consistently** and rarely has bad days, Sortino is high. Upward swings don't hurt you. Only **losses count against you**. This has the HIGHEST weight — avoiding losses is the #1 priority.
• Sharpe ratio (30% weight): Measures returns relative to ALL volatility (up and down). **Consistent small daily gains** = high Sharpe. Wild swings (even if profitable overall) = low Sharpe. Think of it as: how smooth is your equity curve?
• Calmar ratio (30% weight): Total return divided by maximum drawdown (**your worst peak-to-trough drop)**. If you made 10% total but at one point lost 5% from peak, Calmar = 2.0. A single big crash destroys this ratio even if you recover afterward.

NOT LOSING MONEY MATTERS MORE THAN MAKING MONEY.

Key Dates

Date	What happens
March 18-20	Build and test the bot (NOW)
March 21	Round 1 starts — switch to competition API keys, deploy on AWS
March 21-22	Run conservative (50% position sizes) to validate bot works
March 23-27	Full throttle — generate most of our returns
Before March 28	Submit open-source GitHub repo link
March 28-31	Protect gains — tighten stops, reduce sizes
April 2	Finalists announced (top 8 per city)
April 4-14	Final round (if we advance)
April 17	Presentation deck due

THE CORE IDEA — WHAT OUR BOT DOES

Bitcoin doesn't behave the same way all the time:
• Sometimes it TRENDS — steadily climbs or drops for hours. During trends, you want to ride the wave (momentum strategy).
• Sometimes it BOUNCES — goes up-down-up-down in a range. During bouncing, you want to buy at the bottom of the range and sell at the top (mean-reversion strategy).
• Sometimes it goes CRAZY — unpredictable violent swings. During chaos, you want to sit in cash and wait.

Our bot detects which mode the market is in, then applies the correct strategy for that mode. On top of that, an ML model (XGBoost) checks every signal to confirm: 'Is this trade likely to make money after fees?' If the model says no, we skip the trade.

A trade must pass through ALL 7 layers of our pipeline to execute. Most signals die early. That's the point — we only take the highest-conviction trades.

THE FULL FLOW: ONE COMPLETE CYCLE

Here is exactly what happens every 60 seconds, from start to finish:

1. FETCH DATA

Bot calls Roostoo /v3/ticker for BTC_USDT. Gets: LastPrice, MaxBid, MinAsk, Change, Volume.

Bot calls /v3/ticker (no pair) for all coins. Calculates market breadth.

Stores price in local DataFrame. Computes all indicators from stored history.

2. LAYER 1: REGIME CHECK (recalculated every hour)

ATR, ADX, BB width, breadth, Fear & Greed, funding rate → TRENDING / SIDEWAYS / VOLATILE

3. LAYER 2: SIGNAL GENERATION

Based on regime, activate the right indicators. Output: BUY / SELL / HOLD

If HOLD → go back to step 1. Wait 60 seconds.

4. LAYER 3: REVERSAL BLOCKER

Check for extreme move, wide spread, abnormal volume.

If BLOCKED → skip trade, wait 2 cycles. Go back to step 1.

5. LAYER 4: MULTI-TIMEFRAME

Score 1H + 4H + Daily. Sum the scores.

If sum < +2 → skip trade. Go back to step 1.

If 4H is bearish → skip trade. Go back to step 1.

6. LAYER 5: XGBOOST

Feed 10 features to the model. Get probability.

If probability < 0.65 → skip trade. Go back to step 1.

7. LAYER 6: POSITION SIZING

Calculate: quarter-Kelly x regime_mult x timeframe_mult, constrained by drawdown cap and hard limits.

Output: exact dollar amount to trade.

8. LAYER 7: EXECUTE

Place LIMIT order. Set trailing stop. Log everything.

Monitor position until exit (stop-loss, take-profit, or time-exit).

9. REPEAT

Wait 60 seconds. Go back to step 1.

Most cycles, the bot does NOTHING. A signal dies at Layer 1 (wrong regime), Layer 3 (spike trap), or Layer 4 (ML not confident). This is correct behavior. We only want 4-8 trades per day, not 100.

EXPECTED PERFORMANCE

Metric	Conservative	Target	Best Case
10-day return	2-3% ($20-30K)	5-8% ($50-80K)	10-15% ($100-150K)
Sharpe ratio (annualized)	1.0	1.5-2.0	3.0+
Sortino ratio (annualized)	1.2	2.0-2.5	4.0+
Win rate	55%	60-65%	70%+
Max drawdown	3-5%	2-3%	<2%
Trades per day	2-4	4-8	8-12
Average profit per winning trade	0.3%	0.5%	0.8%

Add Later

ML Feature Importance Dashboard (Layer 4 Enhancement)
Kelly Adaptation Logic (Layer 6 Enhancement)
Emergency Kill Switch (Layer 6 Enhancement)

Layers

Layer 1

THE ARCHITECTURE — 7 LAYERS

Every 60 seconds, the bot fetches the current BTC price from Roostoo's /v3/ticker endpoint and stores it in a local DataFrame (building our own price history). It also fetches all coin tickers for market breadth calculation. Then it runs through 7 layers:

LAYER 1: REGIME DETECTION

"What kind of market are we in right now?"

Before making any trading decision, the bot first classifies the market into one of three modes. This determines which strategy we use and how much risk we take.

How it works

The bot calculates 6 inputs and combines them to determine the regime:

INPUT 1 — ATR(14) / Price (Volatility Level)

ATR = Average True Range. It measures how much BTC price moves per candle on average over the last 14 candles. Dividing by price gives a percentage. If BTC typically moves $800 per hour and the price is $80,000, that's 1% volatility. We compare the current ATR/Price against the ATR/Price over the last 14 days.

- Bottom 20th percentile of last 14 days = LOW volatility = SIDEWAYS regime likely
- 20th to 80th percentile = NORMAL volatility = TRENDING regime likely
- Top 20th percentile = HIGH volatility = VOLATILE/DANGER regime

INPUT 2 — ADX(14) (Average Directional Index)

ADX measures how STRONG a trend is, regardless of direction. It gives a number from 0 to 100.

- ADX > 25 = market IS trending (strong directional movement)
- ADX < 20 = market is NOT trending (sideways, choppy)
- ADX between 20-25 = ambiguous

ADX does NOT tell you if the trend is up or down — just whether a trend exists at all.

INPUT 3 — Bollinger BALLS Width haha oomb

Bollinger Bands create a channel around the price: middle = 20-period average, upper = middle + 2 standard deviations, lower = middle - 2 standard deviations. The WIDTH of this channel tells you about volatility:

- Narrow bands (squeeze) = market is quiet, coiling up for a potential big move

- Wide bands = market is volatile, already making big moves
- Expanding bands = volatility is increasing
- Contracting bands = volatility is decreasing

INPUT 4 — Market Breadth

The percentage of ALL coins on Roostoo that are currently in a positive trend (price up in the last hour). We get this by calling /v3/ticker without a pair parameter, which returns data for every listed coin.

- Breadth > 60% = most coins are rising = healthy market = RISK-ON
- Breadth < 40% = most coins are falling = unhealthy market = RISK-OFF

Why this matters: if BTC gives a buy signal but 90% of other coins are crashing, the BTC signal is probably a fakeout. If everything is rising, the signal is high-conviction.

INPUT 5 — Fear & Greed Index (Daily)

A single number from 0 to 100 representing overall crypto market sentiment. Free API, no key needed: https://api.alternative.me/fng/

Score	Meaning	What we do
0-25 (Extreme Fear)	Everyone is panicking and selling	Aggressively buy dips — historically the best time to buy
25-45 (Fear)	Market is nervous	Normal buying allowed, slightly larger positions
45-55 (Neutral)	No strong sentiment signal	Normal operations
55-75 (Greed)	Market is euphoric, people are greedy	Tighten stops, reduce position sizes
75-100 (Extreme Greed)	Peak FOMO, everyone buying	Don't open new positions, prepare to exit existing ones

As of March 2026: Fear & Greed is at 15 (Extreme Fear for 38 consecutive days). This historically precedes massive price bounces.

INPUT 6 — Funding Rate (Every 8 Hours)

In the futures market, every 8 hours, one side pays the other. This tells us where leveraged traders are positioned. Free from Binance API.

- **Very negative funding rate:** Shorts (people betting price goes down) are overcrowded. They're paying longs. When too many people are short, even a small price increase forces them to buy back

(cover), creating a chain reaction called a SHORT SQUEEZE that pushes price up violently. This is a BUY confirmation signal.

• **Very positive funding rate:** Longs (people betting price goes up) are overcrowded. They're paying shorts. When too many people are long, even a small drop causes forced selling, pushing price down more. This is a CAUTION signal — reduce confidence in buy signals.

This data comes from OUTSIDE the Roostoo platform (Binance) but gives us information that no price chart can show — where the leveraged money is positioned.

The Regime Decision

Regime	When	What we do next
TRENDING	ADX > 25 AND volatility is normal AND breadth > 50%	Activate momentum/trend signals (Layer 2). Full position sizing allowed.
SIDEWAYS	ADX < 20 AND volatility is low AND breadth between 40-60%	Activate mean-reversion signals (Layer 2). Half position sizing.
VOLATILE	Volatility in top 20% OR breadth < 30% OR multiple danger signs	Skip almost all trades. Only trade if ALL other layers agree at 90%+. Position sizes at 10% of normal.

The regime is recalculated every hour (not every 60 seconds) because market regimes don't change every minute.

Layer 2

LAYER 2: SIGNAL GENERATION

"Should we BUY, SELL, or do NOTHING right now?"

Based on what regime we're in (from Layer 1), the bot activates DIFFERENT indicators. Using momentum indicators in a sideways market is a recipe for losses. Using mean-reversion indicators in a trending market misses the big move. The regime determines which toolkit we use.

IF REGIME = TRENDING: Use Momentum Signals

Signal A — Donchian Channel Breakout (PRIMARY)

The Donchian Channel draws two lines:

- UPPER line = the HIGHEST price in the last 20 periods (20 hours on 1H candles)
- LOWER line = the LOWEST price in the last 10 periods (10 hours on 1H candles)

When the current price breaks ABOVE the upper line, it means BTC just hit a new 20-hour high. This signals the start of a new uptrend. BUY.

When the current price breaks BELOW the lower line, it means the trend is dying. SELL / EXIT.

This is from the Turtle Trading system (1980s). Richard Dennis recruited complete beginners, gave them these rules, and they made hundreds of millions. The system proves that simple, disciplined rules beat gut feeling. Crypto trends are violent and persistent — Donchian catches them at the start.

Why 20/10? Asymmetric: we use a longer lookback to enter (20 = more conservative, avoids false breakouts) and shorter to exit (10 = exit faster when trend reverses to protect profits).

Signal B — EMA Alignment

Check if three EMAs are stacked in order:

- EMA(9) > EMA(21) > EMA(50) = bullish alignment = confirms uptrend
- EMA(9) < EMA(21) < EMA(50) = bearish alignment = confirms downtrend

EMA = Exponential Moving Average. Like a regular average but gives MORE weight to recent prices. EMA(9) reacts faster than EMA(21) which reacts faster than EMA(50). When the fast one is above the medium one which is above the slow one, it confirms a strong uptrend at all speeds.

Signal C — MACD Histogram Rising

MACD measures momentum. The histogram is the difference between the MACD line and its signal line.

- Histogram going from negative to positive = momentum shifting bullish
- Histogram increasing = momentum accelerating

- Histogram decreasing = momentum fading (early warning to tighten stops)

IF REGIME = SIDEWAYS: Use Mean-Reversion Signals

Signal A — RSI Oversold/Overbought

RSI (Relative Strength Index) gives a score from 0 to 100 measuring whether BTC has been bought too aggressively or sold too aggressively recently.

- RSI(14) < 35 = oversold = price dropped too fast, likely to bounce back = BUY signal
- RSI(14) > 65 = overbought = price rose too fast, likely to pull back = SELL/EXIT signal

Note: We use 35/65 instead of the standard 30/70 because in sideways markets, we want to catch the reversal slightly earlier. In a trending market, RSI can stay above 70 for days — that's why we only use RSI for sideways regimes.

Signal B — Bollinger Band Touch

- Price touches or breaks below the LOWER Bollinger Band = unusually cheap relative to recent history = BUY
- Price touches or breaks above the UPPER Bollinger Band = unusually expensive = SELL/EXIT

In a sideways market, price tends to bounce between the bands like a ping-pong ball. This is the definition of mean-reversion: price went too far from the average and snaps back.

Signal C — Z-Score from Recent Mean

Z-score = (current price - average price) / standard deviation. It tells you how many standard deviations away from 'normal' the current price is.

- Z-score < -1.5 = price is 1.5 standard deviations below normal = very cheap = BUY
- Z-score > +1.5 = price is 1.5 standard deviations above normal = very expensive = SELL

IF REGIME = VOLATILE

Default action: HOLD. Do not trade.

The only exception is if ALL remaining layers (3, 4, 5) pass with maximum scores AND the signal confidence is above 90%. This should almost never happen — that's the point. In volatile markets, the best trade is no trade.

Output of Layer 2: BUY / SELL / HOLD + the direction

Layer 3

LAYER 3: REVERSAL BLOCKER

"Is this a trap? Should we wait?"

This layer prevents the single most common mistake in crypto trading: CHASING. When BTC suddenly pumps 3% in 10 minutes, your indicators will scream BUY. But that pump might already be over — you'd be buying at the top.

How it works

Before entering ANY trade, the reokiersal blocker checks three conditions:

CHECK 1 — Extreme Recent Move

Did BTC price move more than 2% in the last 1-3 candles (intervals)?

- YES = BLOCK. The move already happened. Entering now means you're late. The probability of a pullback is high.
- NO = PASS.

CHECK 2 — Spread Widening

Is the spread (MinAsk - MaxBid) / LastPrice wider than its recent average?

- YES = BLOCK. Wide spreads mean uncertainty and low liquidity. Market makers are pulling back. This is a danger sign.
- NO = PASS.

We calculate spread proxy from Roostoo ticker data: (MinAsk - MaxBid) / LastPrice. Compare to its 20-period average. If current spread > 1.5x the average, block.

CHECK 3 — Abnormal Volume

Is the current volume more than 3x the 20-period average volume?

- YES = BLOCK. Volume spikes often accompany the END of a move, not the beginning. The big players already traded. You'd be the last one in.
- NO = PASS.

If ANY of these three checks trigger: BLOCK the trade for 2 cycles (2 minutes).

After the cooldown, re-evaluate from Layer 2. If the signal is still there after the spike has settled, it's more likely to be real.

Why this matters: In Kireeti's research, this single filter 'often improves live behavior more than adding a fancier model.' It's the simplest layer in our architecture but one of the most valuable for protecting Sortino and Calmar ratios.

Layer 4

LAYER 4: MULTI-TIMEFRAME FILTER

"Does the big picture agree with our trade?"

A buy signal on a 1-hour chart means nothing if the daily chart is pointing down. This layer checks whether SHORT, MEDIUM, and LONG-term views all agree.

How it works

Score three timeframes independently:

Timeframe	What we check	Score
1-Hour (short-term)	EMA(21) slope direction + MACD above/below signal line	+1 if bullish, 0 if neutral, -1 if bearish
4-Hour (medium-term)	EMA(21) slope direction + MACD above/below signal line	+1 if bullish, 0 if neutral, -1 if bearish
Daily (long-term)	Is price above or below EMA(50)?	+1 if above (bullish), -1 if below (bearish)

Add the scores:

Total Score	Action
+3 (all bullish)	PASS — use FULL position size
+2	PASS — use HALF position size
+1 or less	BLOCK — do NOT trade
Negative	BLOCK — definitely do NOT trade

HARD RULE: If the 4-hour timeframe is bearish (-1), NEVER take a buy trade, even if the 1-hour and daily are bullish. The 4-hour trend is our primary directional filter.

Why this works: A study on Bitcoin multi-timeframe strategies showed Sharpe ratio improved from 0.33 to 1.07 when using triple-screen filtering (3 timeframes). It eliminates trades where you're fighting the bigger trend.

Note on building multi-timeframe data: Since Roostoo only gives us current price (no historical candles), we must build our own candle history. Every 60 seconds we store the price. From those stored prices, we construct 1H candles (group 60 readings), 4H candles (group 240 readings), and daily candles. This means the bot needs to run for at least 24 hours before the daily timeframe has enough data. During the first hours, only use 1H and 4H.

Layer 5

LAYER 5: XGBOOST CONFIRMATION

"Does the data actually support this trade?"

The indicators in Layer 2 give us a direction (BUY/SELL). But indicators alone generate many false signals. The XGBoost model acts as a filter: it looks at a broader set of data and asks one specific question:

"Will the forward return over the next N bars be GREATER than our transaction fees + a minimum profit threshold (0.1%)?"

If the model says <65% probability: NO TRADE, regardless of what the indicators say.

What is XGBoost?

XGBoost is a machine learning algorithm. It builds thousands of simple decision trees (if-then rules) and combines their votes. Think of it as: you ask 400 simple advisors, each looking at the data slightly differently, and go with the majority opinion. It's the most proven ML algorithm for tabular data (spreadsheet-like data). Every Kaggle crypto competition winner used it or its cousin LightGBM.

The Features (Model Inputs)

We feed the model exactly 10 features. Alankritha's research + the EE020 report showed that fewer, better features outperform dumping in everything:

1. Log Returns (1, 3, 5, 10 bars)

Instead of raw price, we use log(Price_now / Price_N_bars_ago). This shows if the move is accelerating or slowing. Log returns are better than raw price because they're scale-independent and normally distributed.

2. ATR/Price (Volatility Ratio)

Current ATR divided by current price. Tells the model if current swings are 'normal' or 'chaotic' relative to the price level.

3. RSI(14) + RSI Change Over 3 Bars

Not just the RSI value, but how RSI itself is changing. This captures the 'momentum of momentum.' If RSI is at 40 but RISING fast, that's different from RSI at 40 and falling.

4. MACD Histogram Value

The raw MACD histogram number. Positive and increasing = strong bullish momentum. Negative and decreasing = strong bearish momentum.

5. Bollinger Band %B (Position Within Bands)

%B tells you where price is relative to the bands. %B = 0 means price is at the lower band. %B = 1 means at the upper band. %B = 0.5 means at the middle. Values outside 0-1 mean price broke through the bands.

6. Volume Ratio (Current / 20-period Average)

Is current trading volume unusual? Volume ratio of 2.0 means twice the normal activity. High volume with a price move = the move is 'real.' Low volume move = potentially fake.

7. Cyclical Time Encoding (sin/cos of Hour)

Instead of using 0-23 for the hour of day, we convert it to sine and cosine coordinates on a circle. This tells the model that 11:59 PM (hour 23) is close to 12:01 AM (hour 0), not at opposite ends of a scale. Crypto has known hourly patterns (US market open/close effects).

8. Rolling Autocorrelation

Measures how much current price moves repeat past patterns. High autocorrelation = trending (prices moving in the same direction as before). Low autocorrelation = random noise. This helps the model distinguish real trends from randomness.

9. Market Breadth

Same as Layer 1 — percentage of all coins trending up. Provides context: is BTC's move supported by the broader market or isolated?

10. Spread Proxy

(MinAsk - MaxBid) / LastPrice from Roostoo ticker. Wider spreads = more risk = model should be more cautious.

Model Training

Training data: 90 days of BTC 15-minute candle data (from Binance public API via ccxt library)

Target variable: Binary (1 = forward return > fees + 0.1%, 0 = not)

Validation: Walk-forward (train on older data, test on newer — never look into the future)

Retrain: Every 24 hours with the latest data, so the model adapts to changing conditions

XGBoost parameters: n_estimators=400, learning_rate=0.1, max_depth=4, min_child_weight=3, subsample=0.8, reg_alpha=0.5, reg_lambda=1.0

Output: A probability between 0.0 and 1.0. If probability < 0.65, the trade is REJECTED.

ML PIPELINE (For Layer 5 — XGBoost)

#	Task	What to build	Owner
M1	Feature engineering	Calculate all 10 features from candle data: 1. Log returns (1/3/5/10 bars) 2. ATR/Price (volatility ratio) 3. RSI(14) + RSI change over 3 bars 4. MACD histogram 5. Bollinger Band %B 6. Volume ratio (current/20-avg) 7. Cyclical time (sin/cos of hour) 8. Rolling autocorrelation 9. Market breadth 10. Spread proxy	Pranati
M2	Label creation	For each candle in training data: did the forward return over next N bars beat fees (0.1%) + profit threshold (0.1%)? Binary label: 1=yes profitable, 0=no.	Pranati
M3	Model training	Train XGBoost classifier with params: n_estimators=400, lr=0.1, max_depth=4, min_child_weight=3, subsample=0.8, reg_alpha=0.5, reg_lambda=1.0. Walk-forward validation. Save model to .pkl file.	Pranati
M4	Daily retrain pipeline	Script that runs every 24h: pulls latest price data, re-engineers features, retrains model with expanding window, saves new model file, regenerates feature importance plot.	Pranati
M5	Feature importance plot	After training: plot model.feature_importances_ as horizontal bar chart using matplotlib. Save as feature_importance.png in repo root. For README + presentation.	Pranati

Layer 6

LAYER 6: POSITION SIZING + RISK MANAGEMENT

"How much money do we put into this trade?"

This is the most important layer for winning. The scoring formula punishes drawdowns THREE ways (Sortino, Sharpe, Calmar). Position sizing determines whether one bad trade hurts 0.5% or 15% of our portfolio.

Step A — Base Size: Quarter-Kelly Criterion

The Kelly Criterion is a mathematical formula that calculates the OPTIMAL fraction of your capital to bet, given your win rate and average win/loss sizes.

Full formula: Kelly% = (win_rate x avg_win/avg_loss - (1 - win_rate)) / (avg_win/avg_loss)

Example: 60% win rate, average win is 1.5x average loss:

Kelly% = (0.60 x 1.5 - 0.40) / 1.5 = (0.90 - 0.40) / 1.5 = 0.333 = 33%

But full Kelly is too aggressive — one bad streak and you blow up. So we use QUARTER-Kelly:

Position size = 33% x 0.25 = 8.3% of capital per trade

Quarter-Kelly captures ~50% of optimal growth rate with ~87% less drawdown risk. Professional traders use half or quarter Kelly. We use quarter because the scoring heavily penalizes drawdowns.

The Kelly variables (win rate, avg win/loss) are RECALCULATED every 20 trades based on actual bot performance. So the sizing adapts: winning streak = bigger bets, losing streak = smaller bets.

Step B — Regime Adjustment

Regime	Size Multiplier	Effective Max Position on $1M
TRENDING	x 1.0 (full)	~$250K-350K
SIDEWAYS	x 0.5 (half)	~$150K-200K
VOLATILE	x 0.1 (tiny)	~$50K-100K

Step C — Timeframe Adjustment (from Layer 5)

- Score +3 = multiply by 1.0 (full confidence)
- Score +2 = multiply by 0.5 (half confidence)

Step D — Drawdown Throttle (Circuit Breaker)

This is our insurance policy. It automatically reduces risk when the portfolio is losing:

Portfolio Drop from Peak	What Happens
Less than 2%	Normal operations. No restrictions.
2% to 5%	Max position capped at 25% of capital. Signal score must be >= 70/100.
5% to 8%	Max position capped at 15% of capital. Only A+ setups (score >= 80/100).
8% to 10%	HALT ALL TRADING for 4 hours. After halt, resume with 10% max position.
More than 10%	EMERGENCY MODE. 5% max position. Need 4H trend confirmation. Essentially near-flat.

Why: If the $1M drops to $900K, we have less capital AND the Calmar ratio is already damaged. The ONLY way to recover Calmar is to stop the bleeding and then slowly make it back with consistent small wins. Continuing to trade aggressively while losing is how teams blow up.

Step E — Hard Limits (Never Violated)

- **Maximum position size:** 35% of current capital. Never bet more than a third of everything on one trade.
- **Maximum loss per trade:** 1.5% of current capital. If the stop-loss math says we'd lose more than $15K on a $1M portfolio, reduce position size until the loss fits within 1.5%.
- **Maximum open orders:** 3 simultaneous positions (though with BTC-only trading, this is effectively 1 position at a time).
- **Risk per trade:** 0.5% of equity = $5K max loss per trade at $1M. This is the dollar amount we're comfortable losing on any single trade.

The Sizing Formula (All Steps Combined)

```
final_position_size = min(
        quarter_kelly_size
        x regime_multiplier
        x timeframe_multiplier,
        drawdown_cap,
        35% of capital,
        (0.5% of equity) / (ATR x 1.5)  # volatility-adjusted size
)
```

The final size is the SMALLEST of all these constraints. Multiple safety nets overlapping = extremely hard to blow up.

Layer 7

LAYER 7: EXECUTION + TRADE MANAGEMENT

"Place the order and manage it."

Entry Execution

1. Calculate exact BTC quantity respecting Roostoo's PricePrecision and AmountPrecision from /v3/exchangeInfo

2. Place a LIMIT BUY order at the current bid price (MaxBid from ticker)

3. Wait up to 2 minutes for the order to fill

4. If not filled after 2 minutes: cancel the order via /v3/cancel_order, re-fetch current price, and re-place at updated bid

5. If still not filled after 2nd attempt: abandon this signal and wait for the next cycle

6. Confirm fill via /v3/query_order

Exit Management

Once a position is open, three exit mechanisms run in parallel:

EXIT 1 — Trailing Stop-Loss (ATR-based)

The stop-loss starts at entry_price - 1.5 x ATR(14). As price moves UP, the stop moves up WITH it. It never moves down.

Example on 1H candles with ATR = $500:

- Buy at $80,000. Initial stop = $80,000 - $750 = $79,250.
- Price rises to $81,000. Stop moves up to $81,000 - $750 = $80,250. (Locked in $250 profit)
- Price rises to $82,000. Stop moves to $82,000 - $750 = $81,250. (Locked in $1,250 profit)
- Price drops back to $81,300. Stop stays at $81,250 (doesn't move down).
- Price drops to $81,200. STOP TRIGGERED. Sell at $81,250. Profit = $1,250.

Without a trailing stop, if we had a fixed take-profit at 5% ($84,000), we would have missed the $1,250 profit entirely because price never reached $84K. The trailing stop captures whatever the market gives us.

In SIDEWAYS regime, we use fixed take-profit at 1.0x ATR above entry instead of trailing, because price bounces rather than trends.

EXIT 2 — Time-Based Exit

If a position has been open for 8 hours and the P&L is less than 0.2% (essentially flat), close it. The capital is being wasted sitting in a trade that's going nowhere. Free it up for the next opportunity.

EXIT 3 — Cooldown After Stop-Loss

After any stop-loss is hit, the bot enters a 1-hour cooldown where no new trades can be opened. This prevents 'revenge trading' — immediately re-entering after a loss, which typically leads to another loss. Emotions don't apply to our bot, but the market condition that caused the loss probably hasn't changed in 5 minutes.

Logging (Everything Gets Recorded)

Every trade is logged with ALL context so we can debug and improve:

- Timestamp
- Regime state (Trending/Sideways/Volatile)
- Signal source (which indicator triggered)
- Reversal blocker result (passed/blocked)
- XGBoost probability score
- Multi-timeframe scores (1H/4H/Daily)
- Position size ($ and BTC amount)
- Entry price
- Stop-loss level
- Exit price and exit reason (stop-loss/take-profit/time-exit)
- P&L in $ and %
- Portfolio value after trade

Monitoring & Alerts

- Heartbeat every 5 minutes: bot sends 'I am alive' signal. If it stops, we get alerted.
- Alert on: trade executed, stop-loss hit, drawdown threshold crossed (2%, 5%, 8%), any error or crash
- Daily summary: total trades, win rate, total P&L, current drawdown, current Sharpe estimate

Problems

1. **Cold start** - pre load binance data into the dataframe at startup
2. **SELL not equal Short** - Code treats SELL as an action regardless of position state - every sell signal must first check - Do we even have a position. No position = ignore the signal completely
3. **Breakout Entry Price** - For Donchian breakout signals, place limit order slightly above current price (bid + 0.02%), not at bid. A bid-price limit will never fill and we miss the exact trade our strategy is built for. For mean-reversion signals,bid price is fine.
4. **Faster Stop-Loss Monitoring** - Main signal loop runs every 60 seconds, but stop-loss monitor should run every 10-15 seconds on a separate thread. BTC can drop 3% in 30 seconds during a flash crash. If we only check every 60 seconds, we eat the full loss before reacting.
5. **State Persistence to Disk** - Save critical state to a JSON/CSV file every cycle: current positions, trade history (for Kelly), peak portfolio value (for drawdown), price history DataFrame, cooldown timestamps. On restart, load from disk. Without this, a crash = full amnesia — bot forgets its positions,re-buys BTC it already owns, Kelly resets to defaults.
6. **Partial Fill Handling** - If a limit order only partially fills before we cancel at 2 minutes: if more than 50% filled, treat as valid position and manage normally. If less than 50% filled, sell back immediately then weak signal that couldn't even fill. Log it.
7. **One Position at a Time** - If we already hold BTC and another BUY signal fires, ignore it. Don't pyramid or stack positions. Simplifies stop management and prevents overconcentration. Layer 6 caps at 35% anyway. a second entry would break that limit.
8. **Mixed Data Retraining** - Initial XGBoost trains on 90 days of Binance data. Daily retraining adds Roostoo data. But Roostoo is a mock exchange with different volume/spread behavior. Fix: only retrain on price-derived features (log returns, RSI, MACD, ATR) which transfer between exchanges. Don't trust Roostoo volume features until we have 3+ days of Roostoo-specific data.
9. **Market Breadth Source** - Calculate market breadth from Binance public API, not Roostoo. Roostoo is a mock exchange where some coins have zero volume because no team trades them. Breadth from Roostoo reflects hackathon participant behavior, not actual market health. Binance = real volume, real breadth.
10. **Flexible Competition Timeline** - Days 8-9 "reduce sizes" should be a guideline, not a hard rule. If the regime detector says STRONG TRENDING on Day 9, allow normal sizing. Only apply conservative rules if regime is SIDEWAYS or VOLATILE. The drawdown throttle already handles risk — the timeline shouldn't be a second, dumber risk manager overriding a good opportunity.
11. **Backtest Minimum Before Deploying** - Before going live on March 21, backtest must show: Sharpe > 1.0,max drawdown < 10%, win rate > 50%, net profit > 0 after fees. If any fail, tune and re-test. Do NOT deploy a strategy that loses money in backtesting.
12. **Order Precision Rounding** - Fetch PricePrecision and AmountPrecision from /v3/exchangeInfo on startup and cache them. Round every order's price and quantity to exact decimal places before calling place_order. Wrong decimal = API rejects the order = missed trade.

Task Assignments

THE 7 LAYERS

	Layer	Owner
L1	Regime Detection	Narhen
L2	Signal Generation	Narhen
L3	Reversal Blocker	Alankritha
L4	Multi-Timeframe Filter	Narhen
L5	XGBoost Confirmation	Pranati
L6	Position Sizing + Risk	Kireeti
L7	Execution + Trade Mgmt	Kireeti

DATA PIPELINE (Before layers can work)

#	Task	Owner	What to build
D1	Pull BTC historical data	Pranati	Use ccxt library to download 90 days of BTC/USDT 1H candles from Binance public API. Save to CSV. No API key needed. Feeds XGBoost training + backtester + cold start.
D2	Cold start bootstrap	Narhen	On bot startup, load Binance CSV into DataFrame so all indicators (ATR, ADX, Donchian, EMAs) work immediately on Day 1 instead of waiting 14+ days for data to accumulate.
D3	Fear & Greed fetcher	Alankritha	API call to https://api.alternative.me/fng/ once per day. Parse JSON response, return score 0-100. Feed into Layer 1 regime detection.
D4	Funding rate fetcher	Alankritha	API call to Binance for BTC perpetual funding rate every 8 hours. Parse response, return positive/negative value. Feed into Layer 1.

D5	Market breadth calculator	Alankritha	API call to Binance (NOT Roostoo) for top 20-30 coin prices. Calculate % of coins trending up in last 1 hour. Feed into Layer 1 + XGBoost feature.
D6	Candle builder	Narhen	Store every 60-second Roostoo ticker price in DataFrame. Aggregate into 1H, 4H, Daily candles for multi-timeframe analysis. Must handle gaps if API call fails.

SAFETY LOGIC (Critical fixes — must be coded)

#	Task	What it prevents	Owner
S1	SELL = exit only	Every SELL signal must check: do we have a position? No position = ignore completely. We can't short. Without this, bot tries to sell BTC we don't own = API errors.	Narhen
S2	One position at a time	If already holding BTC and another BUY fires, ignore it. Don't stack positions. Prevents overconcentration. Simplifies stop management.	Narhen
S3	Partial fill handling	After 2-min cancel: >50% filled = keep as valid position. <50% filled = sell back immediately (weak signal). Log it.	Kireeti
S4	Breakout entry price	For Donchian breakout signals: limit at bid + 0.02% (slightly aggressive, ensures fill during fast moves). For mean-reversion signals: limit at bid (price isn't running).	Narhen

S5	Order precision rounding	Fetch PricePrecision + AmountPrecision from /v3/exchangeInfo on startup. Cache them. Round every order's price and quantity before place_order call. Wrong decimal = rejected order.	Alankritha
S6	Emergency kill switch	If drawdown > 15% OR rolling Sharpe (3-day) < -0.5: close ALL positions with market orders, halt 24 hours, alert team. Last line of defense.	Kireeti
S7	Flexible competition timeline	Days 8-9 'reduce sizes' only applies if regime is SIDEWAYS or VOLATILE. If regime = STRONG TRENDING, allow normal sizing. Don't override a good opportunity with a calendar rule.	Narhen

TESTING

#	**Task**	**What to build**	**Owner**
T1	Real-data backtester	Run full 7-layer pipeline on 90 days of Binance historical data. Simulate trades with $1M starting capital, deduct 0.05% per trade (limit fees). Output: total return, Sharpe, Sortino, Calmar, max drawdown, win rate, trade count, equity curve plot.	Kireeti
T2	Backtest validation gate	Before deploying: backtest MUST show Sharpe > 1.0, max drawdown < 10%, win rate > 50%, net profit > 0 after fees. If ANY fails, tune parameters and retest. Do NOT deploy a losing strategy.	Pranati

T3	Paper trading test	Run bot on Roostoo TEST API (not competition API) for a few hours on March 20. Verify: orders place correctly, stops trigger, logging works, no crashes. Then switch to competition API on March 21.	Narhen

Alankritha

Main deliverable: **A trading bot that automatically buys and sells assets.**

Overview of workflow expected:

1. **Fetch data from the Roostoo API**
2. **Analyze price movements**
3. **Decide: BUY / SELL / HOLD**
4. **Send orders to the exchange**

Goal: Make the most money, starting from the 1M portfolio already given to us

Rules & Constraints broken down:

1) No market-making: Can't buy and sell orders simultaneously.
2) No arbitrage: cannot exploit price differences across markets simultaneously
3) Only directional trading - predicting of price direction
4) Spot trading only - buy the asset and own it, price rise and drops will affect you
5) No leverage - cannot borrow funds to amplify trades
6) No shorting - cannot sell assets you do not own

Bot strategy

- Trade a small set of liquid majors + one clean baseline ML model + additional strategies that have crypto specific filters

  Trade choices:
  1) BTC as main traded asset (currently dominates crypto at roughly 56.5%)
  2) ETH as secondary (or optional) traded asset - use it mostly as a confirmation signal
  3) SOL as a context asset, not necessarily always traded

Use ETH and SOL as confirmation features: if ETH and SOL are also strong, BTC momentum is more trustworthy.
OR after building a BTC only bot, we can later use ETH & SOL as context features if there is time.

Behavior summary

BTC

- best liquidity

- strongest institutional flows
- cleanest trend structure
- lowest chaos among majors
- best baseline asset to trade

ETH

- good liquidity
- follows BTC often, but can lag
- useful as “risk-on breadth” confirmation

SOL

- more explosive momentum
- more fakeouts, sharper drawdowns
- useful as a risk appetite signal
- should have stricter entry thresholds if traded directly

Baseline ML model : **XGBoost Classifier**

- Works well on tabular market features
- Nonlinear relationships handled
- Fast to train
- East to retrain everyday

Prediction of future price exactly wont really work, this is the question our code should aim to answer
Will BTC’s forward return over the next N bars be greater than fees + threshold?
Optimize for: strong returns, high Sharpe ratio, controlled drawdown, logical and valid strategy design, clean implementation

Overall architecture

Layer 1: Market Regime detection - what kind of market is this?

- Before any entry signal, detect what kind of market BTC is in.

Regimes: Trending, range-bound, high volatility (unstable) - this should be detected first

Features used for regime detection

- ATR / price (volatility level)
- ADX (average directional index) -is the market trending or not?
- Bollinger Band width (market tight vs expanding) - tight means quiet market
- 50 EMA slope (trend direction)
- recent return autocorrelation (trend vs randomness) - are returns repeating themselves

All of the above are simple to implement in code

Layer 2: Regime-Specific Signal generation - what indicators apply now (based on the conclusion for layer 1)

From the above, activate the indicators that fit the current regime (which one of 3)

If regime = trending, use trend-following indicators

Moving Average crossover

MACD

price above key EMA

breakout above recent local high

positive momentum

If regime = range bound, use mean-reversion indicators

RSI oversold / overbought

Bollinger Band lower/upper touch

z-score distance from recent mean

rejection candles near range edges

If regime = high volatility,

Default action:

HOLD

or allow only very high-confidence setups with reduced size

Layer 3: ML confirmation model (XGBOOST)

Main role of model is to perform binary classification
Answers this one specific question - Is this specific combination of indicators likely to result in a profit greater than my transaction fees?

If the model confidence is low, like less than 70%, then there will be no trade.
It is trained with a "Profit Threshold." It only confirms a trade if it predicts a move large enough to cover the entry fee, the exit fee, and potential "slippage" (price moving while you are trying to buy).
Features to consider implementing as the core ones in the ML model, to filter out false signals
Most impactful

- Log returns - Instead of raw price, use `log(Price_t / Price_{t-n})`. Specifically, include returns for the last 1, 3, 5, and 10 bars. This helps the model see if a move is accelerating or slowing down.
- Volatility ratio - The `Average True Range / Price`. This tells the model if current price swings are "normal" or "chaotic."
- RSI changes - Include the **Change in RSI** over the last 3 bars. This captures the *momentum of the momentum*.

Time Aware features

- Cyclical time encoding (sine/cosine) - Instead of using 0–23 for hours, you convert the time into coordinates on a circle. It tells the model that 11:59 PM (23:59) is only 2 minutes away from 12:01 AM (00:01). Without this, the model thinks they are at opposite ends of a scale.
- Lagged log returns - history of log returns. Allows XGBoost to see acceleration. If the return from 5 bars ago was 0.1% and the current return is 0.5%, the model sees a "rocket ship" pattern forming.
- Rolling Autocorrelation: This measures how much current price moves repeat past patterns. It helps the model distinguish between a Trend (high autocorrelation) and Random Noise (low autocorrelation).
-

Market context features

- Market breadth - The percentage of the top 50 coins that are currently in a positive 1-hour trend.
  If you want to buy BTC, but 90% of other coins are crashing, your BTC signal is likely a "fakeout." If everything is rising, your signal is high-conviction.
- Volatility z-score: A comparison of the current ATR to the average ATR of the last 24 hours.t: If the Z-score is > 2.0, the market is "exploding." The model uses this to lower its probability score and stay in cash to protect your Sharpe Ratio.
- Order flow imbalance: The difference between the volume of "Buy" orders and "Sell" orders sitting in the order book. This is a leading indicator. If there is a massive wall of buy orders, the price is likely to move up soon, even if the current candle is red.

Layer 4: Multi Timeframe Confirmation: Is the big picture aligned?

(same as what's in narhen's tab) - 4 hours trend filter (4h EMA slope, 4 MACD direction, 4h price relative to 50 EMA) - im actually not 100% sure what all that means but it seems like a necessary layer to implement

Layer 5: Trade decision Engine - is this trade good enough?

Hard filters (has to pass)!!

If regime not suitable OR timeframe not aligned OR ml prob < 0.70, this could filter out bad trades automatically

Score the startup - if the prev step is passed, there could be the next step with each criteria getting a weight, and a max score of 100 where 70 min has to be met.

The Volatility Ratio is a technical indicator used to identify when a price move is becoming **extreme** or "unnatural" relative to its recent history.

Layer 6: Position Sizing (how much money do i put into each trade) - How much do we risk?

Risk per trade (0.5%?) - adjust volatility
Position size = capital * risk_per_trade/ stop_distance

Layer 7: trade management - When do we get out?

stop_loss = entry_price - 1.3 * ATR
take_profit = entry_price + 1.5 * ATR
(condition for when to exit)

Layer 8: execution logic - How do we place the order?

To prevent overtrading, avoids chasing, reduces fees

Layer 9: drawdown circuit breaker - Are we in a good/bad patch
a portfolio check at the start of every loop iteration, before any signal is evaluated.
If the portfolio has dropped more than 8% from peak, the bot skips all new trades.

The Sharpe Ratio tells you how much **excess return** you are earning for every unit of **risk** (volatility) you take.

🔄 The Full Autonomous Flow

An **autonomous** bot runs in a continuous loop without you touching it. Here is the "Life Cycle" of one loop:

1. **Data Pulse:** The bot calls the Roostoo `/v3/ticker` API every 60 seconds. It saves this data into a local `pandas` DataFrame.
2. **Layer 1-4 (The Brain):**
    - It calculates indicators (RSI, MACD, ATR).
    - It identifies the **Regime** (Trending vs. Sideways).
    - It asks the **XGBoost model** for a probability score.
3. **Layer 5-6 (The Filter):**
    - It checks the **Volatility Ratio**. If the move is too extreme, it aborts.
    - If all signals agree (Score > 70), it calculates the **Dynamic Position Size** based on current ATR.
4. **Layer 7-8 (The Execution):**
    - It sends a **Limit Order** to the exchange.
    - It starts a "Monitor" thread that watches the price to trigger your **Trailing Stop-Loss**.
5. **Layer 9 (The Safety):**
    - It checks total portfolio health. If the **Drawdown** or **Sharpe** are bad, it flips the "Kill Switch" and pauses for 4 hours.

However it's also good to consider the fact that BTC alone is not very reliable, so adding a ranking layer sort of thing to check with one is the best currency in the market at the current moment could be a good idea - a cross sectional momentum instead of simply sticking to BTC

So before all the layers, the bot pulls a full list of tradable coins from the Roostoo API - filters out dead coins and figure out the top few most liquid assets
For each coin, the bot calculates its relative strength, and ranks them based on recent returns (maybe last 30 mins to 1 hr?)

And then implement the above layers - regime + ML filter etc

Top strategies

1. Regime Detection (ATR + ADX + BB Width)
2. Regime-Gated Signal Switching
3. XGBoost/LightGBM Confirmation Model
4. Reversal Blocker
5. Multi-Timeframe Confluence
6. Donchian Channel Breakout
7. Quarter-Kelly Position Sizing
8. Trailing Stop-Loss (ATR-based)
9. Drawdown Circuit Breaker
10. Fear & Greed Index
11. Funding Rate
12. Limit Orders Only
13. Feature Engineering (log returns, RSI changes, cyclical time, volatility ratio, autocorrelation)
14. Market Breadth
15. Spread Proxy as Feature

Chat Breakdown

Here is a **complete breakdown of the SG vs HK Web3 Quant Trading Hackathon** based on the links you sent and available info. I'll organize it so you understand **(1) what the hackathon is, (2) what you must build, (3) how the competition works, (4) tech stack + APIs, and (5) how to actually win**.

---

1. What This Hackathon Is

The **Singapore 🇸🇬 vs Hong Kong 🇭🇰 University Web3 Quant Trading Hackathon** is a **quantitative trading competition where teams build autonomous trading bots** that trade crypto markets.

Key idea:

> Build a **fully automated trading algorithm** that trades cryptocurrency using quantitative models.

Your bot will:

- Run **24/7**
- Execute trades automatically
- Compete against other bots
- Be evaluated based on **performance + risk metrics**

Participants are mainly from:

Singapore universities

- NUS
- NTU
- SMU

Hong Kong universities

- HKU

- HKUST
- CUHK

The event is organized by **Roostoo Labs** and student quant societies.

Sponsors include major quant firms like:

- IMC Trading
- Optiver
- Cubist (Point72)
- Gondor Capital

These firms often use hackathons like this to **spot talent for internships and trading roles**.

---

2. What You Actually Have to Build

The hackathon problem statement is simple:

> **Build an autonomous trading bot that trades crypto on the Roostoo exchange API.**

Your bot must:

1. Fetch market data
2. Decide when to buy/sell
3. Execute trades automatically
4. Manage risk
5. Run continuously in the cloud

Your system typically looks like this:

Market Data API
↓
Feature Engineering
↓
Trading Strategy
↓
Risk Management
↓
Execution Engine
↓
Roostoo Exchange API

Possible strategies:

- Statistical arbitrage
- Momentum trading
- Mean reversion
- Market making
- Reinforcement learning agents
- ML prediction models
- AI-driven signals

They encourage creativity:

> Classic quant models, AI agents, ML, or rule-based strategies.

---

3. Competition Timeline (2026)

Registration

Feb 23 – Mar 14

Team formation.

Introduction workshop

Mar 13

Explains APIs and setup.

Strategy building phase

Mar 16 – Mar 20

Build your bot.

Round 1 – City Qualifiers

Mar 21 – Mar 31

- Singapore teams compete against Singapore teams
- Hong Kong teams compete against HK teams

Top **8 teams from each city advance**.

Round 2 – Final Round

Apr 4 – Apr 14

- **8 SG vs 8 HK teams**
- Two “funds” representing each city.

Demo & Pitch

Apr 17

Submit pitch deck.

Grand Finale

Apr 17 – Apr 21

Presentation to judges.

---

4. Team Rules

Typical rules:

Team size:

1 – 4 people

Requirements:

- Must be students
- One team per participant
- Code must be **open-source on GitHub**
- Strategy must be original
- Bots must connect to Roostoo API

---

5. Judging Criteria

Two main categories:

1️⃣ Trading Performance

Measured using quantitative finance metrics:

Sharpe Ratio

Risk-adjusted return.

Sortino Ratio

Downside risk adjusted return.

Calmar Ratio

Return vs drawdown.

These metrics ensure:

High return alone is not enough — risk management matters.

Example:

Bot A

Return: 40%
Sharpe: 0.5

Bot B

Return: 20%
Sharpe: 2.1

Bot B might win.

2️⃣ Strategy & Presentation

Judges evaluate:

- strategy logic
- risk framework
- technical architecture
- code quality

---

6. Prize Structure

Two reward types.

Presentation Awards

1st place

$800 USD

2nd place

$500 USD

3rd place

$300 USD

Multiple awards across SG and HK.

Performance Awards

Based on trading metrics.

Top strategies win additional prizes.

---

7. Technical Infrastructure

This hackathon is **not just a coding challenge**.

It requires **production-style quant infrastructure**.

Typical stack:

Programming

Python (most teams use)

Libraries:

pandas
numpy
scikit-learn
ta-lib
ccxt

Deployment

AWS EC2

Your bot runs continuously.

API interaction

Roostoo API

You will use endpoints like:

GET /market-data
POST /order
GET /positions
GET /portfolio

From the API docs you shared.

---

8. What the Roostoo Platform Does

Roostoo provides:

- crypto market data
- simulated exchange
- leaderboard
- order execution system

Bots interact like this:

Bot → Roostoo API
↓
Simulated crypto exchange
↓
Leaderboard ranking

All teams trade on the **same simulated market**.

---

9. Live Competition Mechanics

During the competition:

Bots run **continuously**.

Leaderboard updates in real time.

Metrics tracked:

ROI
Sharpe
Sortino
Drawdown
Trade history

Teams can monitor performance on the **Roostoo dashboard**.

---

10. What Judges Actually Look For

Winning teams usually combine:

Good alpha signal

Example:

crypto momentum signals
volume imbalance
funding rate arbitrage

Strong risk management

Example:

position limits
stop loss
max drawdown guard

Robust execution

Example:

latency handling
retry logic
API error recovery

Clean architecture

Example:

strategy.py
risk_manager.py
execution_engine.py
data_handler.py

---

11. Strategy Ideas (High Probability to Work)

These are common winning approaches.

Momentum

Trade strong trending coins.

Signal example:

price > moving average
volume spike

Mean Reversion

Price returns to average.

Example:

z-score based trading

Cross-coin arbitrage

Trade relationships between coins.

Example:

ETH/BTC spread

Volatility breakout

Trade sudden volatility spikes.

ML prediction

Train models using:

price
volume
order book
volatility

---

12. What Makes This Hackathon Valuable

Major reasons people participate:

Quant trading exposure

You build **real algorithmic trading infrastructure**.

Recruiting

Sponsors include major trading firms.

High performers may get:

- internship interviews
- networking opportunities

Portfolio project

You leave with:

GitHub quant trading bot
quant research
pitch deck

Great for:

quant roles
trading roles
ML roles

---

13. What Most Teams Do Wrong

Typical mistakes:

Overly complex AI models

Simple strategies often outperform.

No risk management

Bots blow up.

No transaction cost modelling

Strategies look good but fail live.

Poor infrastructure

Bots crash.

---

14. What Winning Teams Usually Build

Architecture example:

```
data/
strategy/
execution/
risk/
backtest/
main_bot.py
```

Pipeline:

```
fetch_market_data()
    ↓
generate_signals()
    ↓
risk_check()
    ↓
execute_order()
    ↓
update_portfolio()
```

---

15. The Most Important Thing

This is not just a coding challenge.

It is a **quant research + trading system challenge**.

Winning formula:

```
Good signal
+ good risk management
+ reliable execution
= winning bot
```

Narhen

What: Build an autonomous BTC_USDT trading bot on Roostoo's mock exchange
Capital: $1M starting balance
Constraints: Spot only, no shorting, no leverage, no market-making, no arbitrage - **directional trading only**
Fees: 0.1% taker (market orders), 0.05% maker (limit orders) - **use limit orders**
Judged on: Returns, Sharpe ratio, win rate, drawdown, strategy validity, code quality

Bot model

Bitcoin doesn't behave the same way all the time. Sometimes it trends (steadily climbs or drops for hours). Sometimes it bounces (goes up-down-up-down in a range). Our bot needs to detect which mode the market is in and play differently:
- Trending? → Ride the wave. Buy early, hold, trail a safety net behind the price.
- Bouncing/sideways? → Buy when it dips to the bottom of the range, sell when it hits the top.
- Crazy volatile? → Sit mostly in cash. Don't gamble.

On top of that, an ML model looks at dozens of data patterns and says "I'm X% confident price goes up." We only trade when the math AND the model agree.

Simple things too do
- Check the price
- Ask itself to buy or sell
- Acts on self decision
- Repeat

Voting mechanism
- Need 2 out of 4 advisors to agree to do anything
- If 2+ say BUY → bot buys
- If 2+ say SELL → bot sells
- Otherwise → do nothing (HOLD)

Safety Net (Risk manage)
- If a trade goes 3% against decision, auto-sell and take the small loss before it gets worse.
- If a trade goes 5% in your favor, auto-sell and lock in the profit. no greedy.
- Don't have too many bets running at once, max it up with n orders

Advisor 1: Moving Average Crossover (Trend Follower)
- tracking Bitcoin's average price over the **last 7 minutes** and the **last 21 minutes**
- If the short-term average rises ABOVE the long-term average → "Price is going up!" → **Vote BUY**
- If it drops BELOW → "Price is falling!" → **Vote SELL**

Advisor 2: RSI — Relative Strength Index (Rubber Band)
- Measures: "Has the price gone up too much or fallen too much recently?"
- RSI gives a score from 0 to 100
- Below 30 → "Bitcoin is oversold, it's too cheap, it'll bounce back" → Vote BUY
- Above 70 → "Bitcoin is overbought, it's too expensive, it'll drop" → Vote SELL

Advisor 3: Bollinger Bands (Price Channel)
- Draws a "normal range" channel around the price (average price ± some wiggle room)
- Price hits the bottom of the channel → "It's unusually cheap" → Vote BUY
- Price hits the top → "It's unusually expensive" → Vote SELL

Advisor 4: MACD (Momentum)
- Compares fast momentum vs slow momentum
- If fast momentum overtakes slow → "Price is accelerating upward" → Vote BUY
- If fast drops below slow → "Price is losing steam" → Vote SELL

Layer 1 : Regime Detection ("What kind of market is it right now?")

Simple version (start with this): Use ATR (Average True Range) on 1-hour candles — it measures how much BTC price swings per hour.

ATR Level	Regime	What we do
Low (bottom 20% of last 14 days)	**Sideways**	Mean-reversion: buy dips, sell rips
Medium (20th-80th percentile)	**Trending**	Momentum: ride the trend direction
High (top 20%)	**Volatile/Danger**	Defensive: tiny positions or sit in cash

Hidden Markov Model trained on rolling returns + volatility to auto-detect 3 states. More accurate, harder to build.

Layer 2: Signal Generation ("Should we buy or sell?")

A) ML Model (XGBoost)

- Trained on X months of real BTC 15-minute candle data
- Predicts: "Will price be higher or lower in 4 hours?"
- Uses these features (inputs):
    - RSI(14) and RSI(30)
    - MACD(12,26,9)
    - 30-period momentum
    - Stochastic oscillator
- Bollinger Band position (where is price within the band?)
- Volume ratio (is current volume unusual?)
    - Hour of day, day of week
    - Realized volatility (last 24 hours)
    - Outputs a probability: e.g., "72% chance price goes up"
    - Retrained every 24 hours with newest data so it adapts

B) Multi-Timeframe Confirmation

Timeframe	What it checks	Purpose
4-hour candles	MACD direction	"What's the big trend?" — only trade WITH this trend
1-hour candles	XGBoost + technical signals	Main decision timeframe
15-min candles	Price action	Precise entry timing for limit orders

Note: Never trade against the 4-hour trend. If the 4h MACD says "down," don't buy no matter what the 1h says.

Layer 3: Signal Scoring ("How confident are we?")

Every potential trade gets a score out of 100, Minimum score to trade: 70/100. Below that, we sit in cash.

- XGBoost says >70% probability
- XGBoost says >60% probability
- Regime aligns with trade type
- 4-hour trend agrees
- Volume is 1.5x above average
- 1+ technical indicator confirms

Layer 4: Position Sizing ("How much do we bet?")

This is the most important part for winning. The hackathon judges Sharpe ratio (consistency of returns), not just raw profit. Many small wins >>> a few big gambles.

RULEES

Regime	Max position size	Why
Trending	25-35% of capital ($250-350K)	High conviction, ride it
Sideways	15-20% ($150-200K)	Less certain, smaller bets
Volatile	5-10% ($50-100K)	Dangerous, minimize exposure

Risk per trade: 0.5% of total equity. On $1M, that's $5K max loss per trade. This sounds small but it's what keeps your Sharpe ratio high.

Quarter-Kelly sizing:
If our win rate is 60% and avg win is 1.5x avg loss:
Kelly% = (0.60 * 1.5 - 0.40) / 1.5 = 33%
Quarter-Kelly = 33% * 0.25 = 8.3% of capital per trade
This maximizes long-term growth while minimizing blowup risk.

Layer 5: Entries & Exits

<u>Entries:</u>
- Always use LIMIT orders (0.05% fee vs 0.1% for market orders — saves $500 per $1M traded)
- Place limit at current bid price (for buys)
- If not filled in 2 minutes, cancel and re-place at updated price

<u>Exit:</u>

Regime	Stop Loss	Take Profit
Trending	Trailing stop at 1.5x ATR below highest price reached	No fixed TP let it run, the trailing stop handles it
Sideways	Fixed at 1.5x ATR below entry	Fixed at 1.0x ATR above entry
Volatile	Hard max: never lose more than 1.5% on one trade	

Time-based exits:

- Position flat (< 0.2% P&L) after 8 hours then close it, free up capital
- 1-hour cooldown after any stop-loss hit (prevents revenge trading)

TRAILING

If you buy at $60,000 and price goes to $62,000, your stop-loss moves UP with it. So if it drops back from $62K, you still sell with profit instead of waiting until you're in a loss.

Layer 6: Drawdown Protection ("Don't blow up")

Portfolio drops from peak by	**Action**
< 2%	Normal trading
2-5%	Max position → 25%, need score ≥ 70
5-8%	Max position → 15%, only A+ setups (score ≥ 80)
8-10%	HALT trading 4 hours, then 10% max position
> 10%	Emergency: 5% max position, need 4h trend confirmation

Acts like our insurance. Cause if the 1M blowws up then we have nothing to trade with. More the money more the profit.

Layer 7: Competition Timeline Strategy

Days	**Approach**	**Target**
1-2	50% position sizes, calibrate, gather data	Break even, validate bot works
3-7	Full throttle, normal sizing	Generate 70% of total returns
8-9	Tighten stops by 25%, reduce sizes	Protect gains
10	Close everything by hour 20, no new trades last 8hrs	Don't lose profits

Hopefully if this goes well then the expected performance should be

Metric	Conservative	Target	Best Case
10-day return	2-3% ($20-30K)	5-8% ($50-80K)	10-15% ($100-150K)
Sharpe ratio (annualized)	1	1.5 -2	3+
Win rate	55%	60-65%	70%+
Max drawdown	3-5%	2-3%	<2%
Trades per day	2-4	4-8	8-12

Layer 8: Donchian Channel Breakout (The Backbone)

Turtle Trading

• Track the HIGHEST price in the last 20 hours and the LOWEST price in the last 10 hours
• Price breaks ABOVE the 20-hour high → BUY (new uptrend starting)
• Price breaks BELOW the 10-hour low → SELL (trend ending)
• Position size based on ATR (how volatile the market is)

WHY
crypto trends are violent and persistent. Donchian catches breakouts AT the start, unlike the reactive voting system in the starter bot.

Layer 9: Fear & Greed Index Filter

Free API, no key needed: https://api.alternative.me/fng/
Updates daily. Used as a REGIME FILTER, not a timing tool.

Score	Meaning	Action
0-25 (Extreme Fear)	Panic selling	Aggressively buy dips historically best time
25-45 (Fear)	Nervous market	Normal buying, slightly larger positions
45-55 (Neutral)	No edge	Normal operations
55-75 (Greed)	Euphoria	Tighten stops, reduce position sizes
75-100 (Extreme Greed)	Peak FOMO	Don't open new longs, prepare to exit

Layer 10: Funding Rate (Contrarian Signal)
Futures funding rate tells us what leveraged traders are doing. Free from Binance API, updates every 8 hours.
• Very negative funding = shorts overcrowded = SHORT SQUEEZE incoming → BUY
• Very positive funding = longs overcrowded = correction likely → CAUTION
30 mins to implement. Shows WHERE leveraged money is positioned — something price charts can't show.

Layer 11: Cross-Market Correlation Signals
Gives us a 2-4 hour HEAD START on macro-driven BTC moves:
• DXY (US Dollar Index) dropping → BTC tends to rise
• S&P 500 futures gapping up → BTC follows within hours
• ETH/BTC ratio rising → risk-on mode (bullish for BTC)
Free via Yahoo Finance (yfinance library). Low effort, high info value.

Closing formula overall

0.4 × Sortino + 0.3 × Sharpe + 0.3 × Calmar

Architecture

Architecture

STEP 1 — REGIME CHECK (every hour):

- ATR → Trending / Sideways / Volatile?
- Fear & Greed Index → Risk-on or Risk-off?
- Funding Rate → Overcrowded longs or shorts?

→ Output: "what mode should we be in?"

STEP 2 — SIGNAL GENERATION (every 15 min):

- Donchian Breakout: Did price break 20-hour high? (PRIMARY)
- XGBoost ML model: probability of price going up
- MACD + RSI: technical confirmation
- DXY / S&P: macro market support

→ Output: buy/sell/hold + confidence score

STEP 3 — RISK CHECK (before every trade):

- Quarter-Kelly: how much capital to allocate
- Drawdown throttle: are we in damage control?
- Regime adjustment: reduce size if volatile

→ Output: "trade X amount" or "don't trade"

STEP 4 — EXECUTE (when score ≥ 60):

- Place LIMIT order (0.05% fee)
- Set trailing stop-loss (ATR-based)
- Log everything
- Wait for next cycle

brainstorm

Bitcoin doesn't behave the same way all the time. Sometimes it **trends** (steadily climbs or drops for hours). Sometimes it **bounces** (goes up-down-up-down in a range). Our bot needs to detect which mode the market is in and play differently:

- **Trending?** → Ride the wave. Buy early, hold, trail a safety net behind the price.
- **Bouncing/sideways?** → Buy when it dips to the bottom of the range, sell when it hits the top.
- **Crazy volatile?** → Sit mostly in cash. Don't gamble.

On top of that, an ML model looks at dozens of data patterns and says "I'm X% confident price goes up." We only trade when the math AND the model agree.

From the previous HK Roostoo hackathon (105 teams), the composite score was:
0.4 × Sortino + 0.3 × Sharpe + 0.3 × Calmar
What this means:
• Sortino (40% weight) = returns vs DOWNSIDE volatility only. Avoiding big drops is #1 priority.
• Sharpe (30% weight) = returns vs total volatility. Consistent small wins beat occasional big wins.
• Calmar (30% weight) = returns vs maximum drawdown. A single 15% drop tanks this even if you recover.
Bottom line: NOT LOSING MONEY matters MORE than MAKING MONEY. A team with 5% return and 2% max drawdown beats a team with 15% return and 10% max drawdown.

"Adaptive Regime Momentum" (ARM)

What Won	What Lost
Simple models (LightGBM, Ridge Regression, XGBoost)	Complex deep learning (LSTM, Transformers, RL)
Feature engineering (dozens of rolling window features)	Fancy model architecture
Strict risk management, mandatory stop-losses	"Let it ride" mentality
Adapting strategy during competition	Set-and-forget
Capital preservation first	Chasing maximum returns
Classic indicators (RSI, MACD) with strict rules	Overcomplicated signal logic
Using multiple uncorrelated data sources	Only using price data

GitHub

https://github.com/freqtrade/freqtrade-strategies - crypto strategies
https://github.com/asavinov/intelligent-trading-bot - ML signal aggregation for crypto
https://github.com/CyberPunkMetalHead/Binance-News-Sentiment-Bot - News sentiment trading
https://github.com/hackingthemarkets/sentiment-fear-and-greed - Fear & Greed backtesting
https://github.com/TimoDiehm/imc-prosperity-3 - 2nd place IMC clean strategy code
https://github.com/roostoo/Roostoo-API-Documents - Official Roostoo API docs

KEY GITHUB REPOS

• mouadja02/bitcoin-technical-indicators-dataset: 70+ pre-calculated BTC indicators, daily updated
• coderback/drw-crypto-market-prediction: DRW competition — Ridge Regression beat Random Forest and Gradient Boosting

Terms

Sharpe ratio

(average return) / (how much returns jump around).
Higher = better. Above 1.0 is good, above 2.0 is great.

Sortino ratio
Same as Sharpe BUT it only cares about DOWNWARD jumps.

Calmar ratio
total return divided by your worst drop.

Kireeti

Top Strategies

For this hackathon, I'd implement **three strategies only**:

1. Cross-sectional momentum ranker
Score every tradable coin each cycle, rank them, and only trade the top 2–5 names. This matches the Roostoo setup better than a single-coin bot because the platform is built around live crypto trading bots on the mock exchange, and the API lets you pull all listed tickers at once from `/v3/ticker` when `pair` is omitted. It also exposes spread-like and turnover-like fields (`MaxBid`, `MinAsk`, `LastPrice`, `Change`, `CoinTradeValue`, `UnitTradeValue`) that are ideal for ranking and filtering.

2. Regime-gated trend following
Only let the momentum model trade when the market is favorable. Use simple regime flags like breadth, average 24h change, realized volatility from your stored snapshots, and spread widening. When regime is weak, cut size or go flat. This is the best way to balance leaderboard return with Sharpe/Sortino-style risk-adjusted behavior rather than just maximizing raw PnL. The Roostoo exchange info and ticker fields are enough to build this without needing heavy external data.

3. Short-horizon reversal blocker
Do not chase every spike. If a coin just had an extreme 1–3 interval move, spreads widened, and turnover looks abnormal, block fresh entries for a cycle or two. In crypto, this single filter often improves live behavior more than adding a fancier model. Since Roostoo supports both `MARKET` and `LIMIT` orders and enforces pair-specific precision and minimum-order rules, a blocker also reduces bad fills and churn.

Best model stack

Given your time limit, the best stack is:

- **Tier 1:** rule-based regime filter
- **Tier 2:** LightGBM/XGBoost or logistic-regression classifier for next-horizon outperform probability
- **Tier 3:** rules for spread, liquidity, cooldown, and position sizing

I would not make LSTM/GRU the primary live engine unless you already have them working. They can be strong, but tree models or even a clean linear classifier will get you deployed faster and are easier to debug.

That said, your uploaded EE020 report is still very useful: it found that **standard inputs + technicals + fundamentals** beat simpler combinations, while sentiment hurt predictive performance, and their final best feature set concentrated on a compact core instead of using everything.

What to reuse from your EE020 report

Your report is reusable as a **methodology template**, not as a direct stock-to-crypto model. The most valuable parts are:

- **Use OHLCV-style baseline first.** Your report used standard OHLCV as the neutral benchmark and only then tested added components. Do the same with Roostoo ticker snapshots.

- **Keep TA as the core signal layer.** The report leaned on MA, RSI, Bollinger Bands, MACD, and related momentum/trend features; those are directly portable to crypto if you build a rolling snapshot store.

- **Use FA as a moderating filter, not a trigger.** In the stock project, FA helped when combined with TA, but the team explicitly noted it works more as a filter than a direct buy/sell signal. For crypto, replace “fundamental analysis” with crypto-native slow filters like turnover rank, market-cap bucket if available, spread quality, or external funding/open-interest/on-chain proxies from the hackathon data pack.

- **Use compact feature selection.** The report’s top features ended up being a small set: OHLCV plus MA30, RSI, BB upper/lower, and valuation-derived deviations. That supports a hackathon-friendly approach: fewer, better features.

Exactly how to adapt the report to the hackathon

Map the report’s stock pipeline to Roostoo like this:

- **Standard inputs** → your own rolling history built from `/v3/ticker`

- **TA indicators** → MA ratios, RSI, Bollinger position/width, return ranks, realized vol

- **FA layer** → replace stock intrinsic-value features with crypto-native “slow state” features:

    - turnover percentile

    - spread percentile

    - universe-relative strength
    - rolling volatility percentile
    - external funding / open-interest / on-chain features only if the hackathon data pack gives them

- **SA layer** → keep off-model, maybe display only
- **Backtest discipline** → preserve the report's time-aware evaluation mindset; don't trust random train/test splits for sequential data.

Best practical strategy recipes

Here are the three deployable recipes I'd prioritize:

A. Momentum basket

- Build 5m/15m/30m snapshot history
- Features: 3, 6, 12, 24 interval returns; MA gap; RSI; BB position; spread proxy; turnover rank
- Predict next-horizon outperform probability
- Buy top 2–3 coins only when regime is risk-on
- Exit when rank drops, stop hits, or regime flips
  This is the highest-probability hackathon strategy.

B. Breakout-with-confirmation

- Only enter when price is near rolling highs, BB width is expanding, turnover is above median, and spread is acceptable
- Filter with regime and cooldown
- Better when the market is trending strongly
  This is simpler than full ML and can be combined with the model score.

C. Relative-strength + mean-reversion blocker

- Long the strongest names
- But skip names that just had an unsustainably sharp short-term move
- This usually improves risk-adjusted performance versus pure chase-momentum

Best way to reuse the attached report in your pitch

Your report also helps with the **story** you tell judges. You already have a documented workflow showing:

- benchmark-first evaluation,
- feature ablation/selection,
- backtesting over random-split metrics,
- model comparison,
- dashboard transparency with confidence, feature importance, and backtest plots.

That is great hackathon material. Reframe it as:
"we reused a proven research workflow from a prior ML trading project, kept the parts that transfer to crypto, and removed the parts that don't."

My blunt recommendation

For this hackathon, do **not** build:

- sentiment-heavy bot
- deep-learning-first bot
- complicated multi-agent AI trader
- stock-style intrinsic value engine for crypto

Build:

- **cross-sectional momentum**

- **regime filter**
- **reversal blocker**
- **strict execution/risk rules using Roostoo exchangeInfo + balance + place_order**

That gives you the best odds of shipping something live and competitive in time. Your EE020 report is most reusable as a **research and evaluation framework**, with TA + compact feature selection kept, sentiment dropped, and FA replaced by crypto-native slow filters.

I can turn this into a **final hackathon blueprint** next: feature schema, model target, rebalance logic, and order rules.

Best strategy stack for this hackathon

1) Core strategy to build

Cross-sectional momentum bot with regime filter + risk governor

Why this fits the hackathon:

- You can trade **any listed coin**, so ranking the universe is better than building a single-coin bot.
- Bots run **24/7 on AWS**, but High Frequency trading **HFT is not supported** and requests can be rejected if you exceed the limit, so you want a **low-frequency, high-conviction** strategy.
- Roostoo gives you `/v3/ticker`, `/v3/exchangeInfo`, `/v3/place_order`, so a momentum/ranking system is much faster to deploy than a market-making or order-book-heavy setup.

2) Lean high-yield action plan

A. Universe filter

- Pull `/v3/exchangeInfo`
- Keep only pairs with `CanTrade=true`
- Exclude pairs with weak liquidity / wide spread / tiny order flexibility

- Respect `PricePrecision`, `AmountPrecision`, `MiniOrder` at execution time

B. Feature engine

Use only features you can build reliably from Roostoo ticker snapshots:

- 1, 3, 5, 10 interval returns
- price vs rolling mean
- rolling volatility
- spread proxy = `(MinAsk - MaxBid) / LastPrice`
- turnover / trade-value strength
- cross-sectional rank of short-term return
- market breadth = % of coins up in last k intervals

C. Tier 1: Regime filter

Three states:

- `RISK_ON`: breadth good, vol moderate, spreads normal
- `NEUTRAL`: mixed market
- `RISK_OFF`: vol spike, breadth collapse, spreads widen

Actions:

- `RISK_ON`: allow full trading
- `NEUTRAL`: half size, fewer positions
- `RISK_OFF`: no new longs, reduce exposure

D. Tier 2: ML ranking model

Best practical model:

- **LightGBM / XGBoost classifier or ranker**
  Fallback:

- **Logistic regression**

Target:

- predict whether a coin will be in the **top third of forward returns** over the next few snapshots

Trading rule:

- every rebalance, rank all tradable coins
- hold only **top 2–3 coins**
- require minimum confidence threshold

E. Tier 3: Risk governor

- max 2–3 open positions
- max per coin weight cap
- volatility-scaled sizing
- hard stop loss
- trailing stop
- daily drawdown kill switch
- cooldown after stop-out

F. Execution

- Start with **MARKET orders only**
- Rebalance **once per minute or slower**
- No fancy order logic unless fully tested

- Log every signal, fill, position, and equity change

Best concrete strategies to implement

Strategy 1 — Cross-sectional momentum

Buy the strongest-ranked coins in the universe.

Best for: return + simple explainability
Why: hackathon lets you trade all listed coins, so relative strength is a direct edge.

Strategy 2 — Regime-gated momentum

Only run momentum aggressively when the whole market environment is favorable.

Best for: Sharpe / Sortino / Calmar
Why: avoids forcing trades in choppy markets.

Strategy 3 — Momentum + reversal blocker

If a coin spikes too hard in one interval and spread blows out, block fresh entries for 1–2 cycles.

Best for: avoiding bad chase entries
Why: improves risk-adjusted performance with very little code.

Hackathon takeaway:
Use the same principle:

- keep the production bot feature set tight
- prioritize features that improve signal quality and risk control

Key selling points for your pitch

- **Built for Roostoo constraints**: low-frequency, robust, rate-limit-safe strategy.
- **Trades the full universe**, not just one coin.
- **Multi-tiered ML pipeline**: regime filter → ML ranker → risk governor.

- **Balanced for all metrics**: not just raw return; explicitly optimized for drawdown and risk-adjusted returns.
- **Production-minded execution**: precision-safe orders, kill switches, logging, restart-safe design.
- **Lean and deployable in 2 days**: high probability of finishing and staying live.
- **Explainable**: every trade can be traced to rank score, regime state, and risk checks.
- **Iteratable**: easy to improve after first deployment without rewriting the architecture.

Best one-line positioning

“A cross-sectional crypto momentum bot with regime-aware exposure and strict risk controls, designed for Roostoo’s live, low-frequency multi-asset environment.”

I can turn this into a **1-page execution checklist** for tonight and tomorrow.

Tab 26

pranati

Maams enna inga vandhu romance panringa
neeyum pannu
to maams: ilyyyyy mwahhh;
Bro yenaku yenga,
dm vaa <3 xD

I love you

Let me fetch the API docs and slides first before giving you proper advice!Okay now I have the full picture! Let me break everything down properly 😊

What the API Actually Gives You 🔍

From the docs, Roostoo gives you:

Data you can GET (fetch):

- Current BTC price (ticker) — `LastPrice`, `Change` (24h %), volume
- Your wallet balance
- Your order history

Actions you can POST (do):

- Place a BUY order
- Place a SELL order
- Cancel an order

⚠️ **Important limitation:** Roostoo only gives you the **current price** — it does NOT give you historical candles or OHLCV data! So your bot needs to **collect and build** its own price history over time, OR fetch historical data from somewhere else like Binance's free public API.

Now — What Strategies Actually Make Sense? 🧠

Given the competition runs **March 21–31 (10 days)**, scored on **Sortino, Sharpe, Calmar**, trading **4H timeframe** on Bitcoin, here's my honest take:

The Scoring Tells You Everything 🎯

Sortino (40%) = punishes LOSSES heavily
Sharpe (30%) = punishes ANY volatility
Calmar (30%) = punishes big drawdowns (crashes)

This means **don't be greedy, don't crash** is MORE important than making the most money. A bot that makes steady 15% beats one that makes 30% but crashes 20% in between.

Strategies That Match This 💡

Strategy 1 — Trend Following with EMA (safe, reliable)

- Use EMA 21 and EMA 50 on 4H
- Only trade in the direction of the trend
- Keeps you OUT of market during choppy periods = fewer losses = better Sortino

Strategy 2 — RSI Mean Reversion (good for ranging markets)

- RSI below 35 on 4H → BUY
- RSI above 65 on 4H → SELL
- Works when BTC is moving sideways

Strategy 3 — Combined Filter (what your team should do)

EMA tells you: what direction is the trend?
RSI tells you: is NOW a good time to enter?
Only trade when BOTH agree → fewer bad trades

Strategy 4 — ATR-based Stop Loss (critical for Calmar score)

- ATR measures how wild BTC is right now
- Set your stop loss as 1.5x ATR below entry
- This directly protects your Calmar score by limiting max drawdown

---

What YOUR Role Is Specifically 📋

Since you're doing **research + features for XGBoost**, here's your concrete task:

1. Get BTC 4H historical data
   → from Binance free API or Kaggle

2. Calculate these features:
   → EMA 9, EMA 21, EMA 50
   → RSI 14
   → MACD
   → ATR 14
   → Volume ratio (current vs 20-period average)
   → % price change over last 1, 6, 24 candles

3. Label each candle:
   → Did price go UP or DOWN in the next 4H?
   → That's your BUY/SELL label for XGBoost

4. See which features best predict the label
   → That tells you which strategies to use!

What to Search Right Now 🔎

In this order:

1. `"Binance API BTC 4H historical data Python"` — free historical data
2. `"ta library Python crypto indicators"` — calculates RSI, EMA, MACD in 3 lines
3. `"XGBoost binary classification trading tutorial"` — how to set up the model
4. `"ATR stop loss strategy crypto"` — understand risk management for Calmar score

Does this feel clearer now? Your role is basically: **get historical BTC 4H data → calculate features → figure out which ones predict price movement → hand that to your XGBoost teammate!** 😊

Final 4 strategies

Here you go! 📝

Strategy 1 — Regime Detection + Adaptive Switching

Core idea: Detect what "mode" the market is in before trading.

The bot first classifies the market into trending up, trending down, or ranging sideways using ADX indicator or HMM model. Then it automatically switches to the right strategy for that mode — momentum strategy when trending, mean reversion when ranging, and does nothing during extreme volatility.

Strategy 2 — LLM Sentiment + Technical Fusion

Core idea: Combine news sentiment with chart signals before trading.

Every 4 hours, fetch latest Bitcoin news headlines from CryptoPanic API. Send them to Claude/GPT to score sentiment from -1 (very bearish) to +1 (very bullish). Combine that sentiment score with technical indicators (RSI + EMA). Only trade when BOTH news sentiment AND technicals agree on the same direction.

Strategy 3 — Kelly Criterion Position Sizing

Core idea: Mathematically calculate HOW MUCH to bet, not just when.

After every 20 trades, calculate your bot's win rate and average win/loss ratio. Feed into Kelly formula: `K% = Win Rate - (Loss Rate / Win Loss Ratio)`. Use that percentage as your position size. High confidence + good track record = bigger bet. Losing streak = smaller bet. Self-adjusting risk management.

Strategy 4 — Multi-Timeframe Confluence

Core idea: Only trade when short, medium AND long term charts all agree.

Before every trade, check 3 timeframes — 1H, 4H, 1D. Score each as +1 (bullish), 0 (neutral), -1 (bearish). Add them up. Score of +3 = strong buy full size. +2 = moderate buy half size. Anything else = hold and wait. Eliminates weak signals completely.

Short enough to note down? 😊

for my dumbfuck brain

PLS LEARN ALL THIS ASAP TO NOT BE A DUMBFUCK

- (tick each one once done)
- Doubts in other tab per chapter

CHAPTER 1 — What Is Trading? (5 mins)

You have $1000. Bitcoin costs $80,000 per coin. You can buy a fraction — like 0.01 BTC for $800. If price goes to $88,000 you sell → you made $80 profit. If price drops to $72,000 you sell → you lost $80.

That's it. Trading = buy low, sell high.

Your bot does this automatically without you touching anything.

CHAPTER 2 — Price Data (5 mins)

Every hour Bitcoin has one "candle" — 5 numbers:

Open = price when hour started
High = highest price that hour
Low = lowest price that hour
Close = price when hour ended
Volume = how much BTC was traded

Green candle = Close was HIGHER than Open (price went up that hour) 📈 **Red candle** = Close was LOWER than Open (price went down that hour

Why Close Price Matters Most

Out of the 5 numbers, **Close is the most important** — almost every indicator (RSI, EMA, MACD) is calculated using Close prices. When people say "Bitcoin is at $80,000" they mean the last Close price.

Your bot stores these every 60 seconds and builds its own history because Roostoo only gives current price.

Why Volume Matters

Price goes UP + Volume HIGH = real move, people are genuinely buying ✅

Price goes UP + Volume LOW = suspicious, could be fake move ⚠️

Price crashes + Volume HIGH = real panic selling, get out ❌

Price crashes + Volume LOW = probably just noise, ignore it

Volume confirms whether a price move is real or fake. Your Layer 3 Reversal Blocker uses this exactly.

How Your Bot Builds Its Own Candles

Since Roostoo only gives current price every 60 seconds, your bot manually constructs candles:

Every 60 seconds → store: timestamp + LastPrice + Volume

After 60 readings → you have 1 hour of data

Group them into one 1H candle:

Open = first price in that hour

High = highest price in that hour

Low = lowest price in that hour

Close = last price in that hour

Volume = sum of all volume that hour

After 240 readings → you have one 4H candle

After 1440 readings → you have one Daily candle

Important Limitation To Know

Because your bot builds its own history from scratch:

Bot starts running → 0 candles

After 1 hour → first 1H candle ready

After 4 hours → first 4H candle ready

After 24 hours → first Daily candle ready

After 14 days → enough history for ATR(14) to be reliable

This means **your bot is essentially blind for the first few hours.** Your architecture already handles this — during early hours only use 1H signals, add 4H after 4 hours, add Daily after 24 hours.

Timeframes Simply Explained

1H candle = zoomed IN = short term picture = noise + signal mixed

4H candle = middle view = medium term = clearer trends

1D candle = zoomed OUT = big picture = very clear trends

Like Google Maps:

1H = street level view

4H = neighbourhood view

1D = city view

Your bot checks all three before trading — that's your Multi-Timeframe layer.

CHAPTER 3 — The Indicators (15 mins)

These are just **math formulas applied to price data.** Nothing magical.

ATR — Average True Range Measures how much BTC moves per candle on average. High ATR = wild market. Low ATR = calm market. Your bot uses it to detect dangerous conditions and set stop losses.

Step 1 — True Range (TR) for each candle:

TR is the *largest* of these three values:

- High − Low (today's range)
- |High − Previous Close| (gap up scenario)
- |Low − Previous Close| (gap down scenario)

You take the max because sometimes price gaps overnight, meaning yesterday's close to today's high/low captures more "true" movement than just the candle body.

Step 2 — Average the last N true ranges:

ATR(14) = average of the last 14 TR values. The widget uses a simple average for clarity, but in practice most platforms use Wilder's EMA (which weights recent candles more heavily).

ADX — Average Directional Index Measures if market is trending or ranging. Gives 0-100.

ADX > 25 = trending (price moving in clear direction)
ADX < 20 = ranging (price bouncing sideways)

Doesn't tell you UP or DOWN — just whether a trend EXISTS.

Bollinger Bands A channel drawn around price.

Narrow channel = calm, about to make big move so buy
Wide channel = already moving wildly so dont buy

Your bot uses width to detect volatility regime.

RSI — Relative Strength Index Measures if BTC is overbought or oversold. Gives 0-100.

RSI < 30 = oversold = price probably bounces UP soon
RSI > 70 = overbought = price probably drops DOWN soon

Lots of sellers = price goes DOWN (too much supply)
No sellers left = price goes UP (no more supply)
Lots of buyers = price goes UP (too much demand)
No buyers left = price goes DOWN (no more demand)

RSI doesn't predict the future — it just tells you when a move is nearly EXHAUSTED and ready to reverse. Buy when selling is exhausted (< 30), sell when buying is exhausted (> 70). 😊

EMA — Exponential Moving Average Average price over last X candles, but recent candles count more.

Price above EMA = uptrend
Price below EMA = downtrend
Short EMA crosses above long EMA = BUY signal (Golden Cross)
Short EMA crosses below long EMA = SELL signal (Death Cross)

MACD Two EMAs fighting each other. Shows momentum direction.

MACD above signal line = bullish momentum
MACD below signal line = bearish momentum

CHAPTER 4 — The 3 Market Regimes (5 mins)

Bitcoin only ever does one of three things:

TRENDING = price moving clearly up or down
→ use EMA + MACD signals
→ full position size allowed

SIDEWAYS = price bouncing between two levels
→ use RSI signals
→ half position size

VOLATILE = price jumping wildly unpredictably
→ don't trade
→ tiny position size only

Your Layer 1 detects which regime using ATR + ADX + Bollinger Bands + more.

CHAPTER 5 — Orders (5 mins)

When your bot wants to buy or sell it places an ORDER on Roostoo:

MARKET order = buy RIGHT NOW at current price
fast but you pay slightly more

LIMIT order = "I'll only buy if price reaches $X"
slower but you get exact price you want

Your bot uses LIMIT orders to avoid overpaying.

CHAPTER 6 — Risk Management (10 mins)

This is the most important part — how to NOT lose everything.

Stop Loss "If price drops X amount after I buy → automatically sell and cut losses"

You buy BTC at $80,000
Stop loss set at $78,800 (1.5 × ATR below entry)
Price drops to $78,800 → bot automatically sells
Maximum loss = controlled

Take Profit "If price rises X amount → automatically sell and lock in gains"

You buy at $80,000
Take profit at $82,000 (2.5 × ATR above entry)
Price hits $82,000 → bot sells, profit locked in

Position Sizing — Kelly Criterion Don't bet everything on one trade. Your bot calculates:

Winning 60% of trades + average win is 1.5x average loss
→ Kelly says bet 8% of portfolio per trade
→ Even if you lose, you still have 92% left

Drawdown How much your portfolio dropped from its highest point.

Portfolio was $1,000,000
Now it's $900,000
Drawdown = 10%

Your bot automatically reduces trading when drawdown gets too high.

CHAPTER 7 — The Scoring (5 mins)

You're judged on 4 things:

Portfolio Return = did you make money? Simple.

Sharpe Ratio = return divided by ALL volatility Higher = better. Means you made money without wild swings.

Sortino Ratio = like Sharpe but only counts BAD volatility (losses) Higher = better. Going UP doesn't hurt your score, going DOWN does.

Calmar Ratio = return divided by your worst drawdown Higher = better. Means you made money without big crashes.

So your bot needs to:
Make money ✅
Not swing wildly ✅
Not have big losing streaks ✅
Not crash badly ✅

CHAPTER 8 — How YOUR Bot Works (5 mins)

Every 60 seconds:

1. Fetch BTC price from Roostoo
2. Layer 1: What regime are we in? (trending/sideways/volatile)
3. Layer 2: Generate signal (buy/sell/hold?)
4. Layer 3: Any danger signs? (spike/weird volume?)
5. Layer 4: Multi-timeframe agree? (1H + 4H + Daily)

6. Layer 5: XGBoost ML model confident? (>65%?)
7. Layer 6: How much to buy? (Kelly formula)
8. Layer 7: Place order + set stop loss + log everything

Most cycles the bot does NOTHING — signal dies at an early layer. That's correct behaviour. You only want 4-8 quality trades per day.

What To Google To Learn More

Just these, in order:

1. `"candlestick chart explained for beginners"`
2. `"RSI indicator explained simply"`
3. `"what is a moving average trading"`
4. Go to **TradingView.com** → search BTC/USD → turn on RSI and EMA and just watch for 20 mins

That's genuinely all you need to not feel lost tonight. You'll understand what's going on even if you're not the one coding it 😊

Doubts

Doubts

- So what abt before the 1h/4h mark? Since theres no data wont we be at a disadvantage? Compared to someone who does 30 min/40 min candles

Tab 25

LAYER 5: XGboost

Step 1:

Download binance data

What we're fetching:

Every 15 minutes for 90 days:
Open, High, Low, Close, Volume

90 days × 24 hours × 4 candles per hour
= 8,640 candles total

We use requests

Summary:

data_fetcher.py does ONE job:
Connect to Binance → download 90 days of BTC 15-minute candles → save as CSV file We only run this ONCE to get training data
Output: btc_data.csv (8,640 rows of OHLCV data)

Step 2: feature engineering

Raw data (what we have):
timestamp, open, high, low, close, volume

After feature engineering (what we need):
timestamp, open, high, low, close, volume,
log_return_1, log_return_3, log_return_5, log_return_10,
atr_ratio, rsi, rsi_change,
macd_histogram, bb_percent_b,
volume_ratio, hour_sin, hour_cos,
autocorrelation, breadth, spread_proxy

What each feature does:

Log Returns → how much price moved (1, 3, 5, 10 candles ago)
ATR/Price → how wild is market right now vs price level
RSI → overbought/oversold (you know this already!)
RSI Change → is RSI itself rising or falling?

MACD Histogram → momentum building or fading?
BB %B → where is price within bollinger bands?
Volume Ratio → is volume unusual right now?
Hour sin/cos → what time of day is it? (cyclical)
Autocorrelation→ is price trending or random noise?
Breadth → placeholder (we get this live from Roostoo)
Spread Proxy → placeholder (we get this live from Roostoo)

Last 2 (breadth and spread) come from Roostoo API live — we can't calculate them from historical Binance data. So we'll add dummy values for training and replace with real values when live.

Extras

Cross-Sectional Momentum — Full Explanation

What It Is (Simple)

Instead of only trading BTC, the bot looks at ALL 67 coins on Roostoo every cycle and asks: **"Which coins are going up the fastest RIGHT NOW?"** Then it buys the top 2-3 strongest coins and ignores the rest.

Think of it like picking players for a team — you don't pick one player and hope they perform. You look at everyone's stats, pick the best performers, and swap them out when someone else gets hot.

How It Works Step by Step

1. Every cycle, fetch prices for all 67 coins from /v3/ticker
2. Calculate each coin's return over the last 1-7 days
3. Rank them: #1 = strongest, #67 = weakest
4. Buy the top 2-3 coins
5. Next cycle: re-rank. If a coin drops out of the top 3, sell it and buy the new top performer
6. Repeat

Does It Actually Win? The Research Says...

YES, but with a massive caveat:

Scenario	BTC-Only	Cross-Sectional Momentum
Bull market	BTC returned 97%	Top coins returned 69%. BTC wins.
Bear market	BTC dropped -37.8%	Top coins dropped only -2.35%. Momentum wins MASSIVELY.
Sharpe ratio	~0.84	1.1 - 1.6
Drawdown protection	None (rides the crash)	Rotates away from losers automatically

The key insight: Cross-sectional momentum doesn't necessarily make MORE money than BTC in a bull run. But it **loses MUCH LESS in a downturn**. And since our scoring is 0.4 Sortino + 0.3 Sharpe + 0.3 Calmar — all of which PUNISH drawdowns — this is exactly what we want.

Did Anyone Win a Competition With This?

Honestly: no clear evidence of a cross-sectional momentum strategy winning a major crypto trading competition. Winners tend to be:

- High-conviction directional bets (WEEX $600K winner)
- ML-based systems (Kaggle winners)
- Market-making strategies (IMC Prosperity winners — but we can't do that)

However: Commercial firms using cross-sectional momentum in crypto (Stoic AI, Unravel Finance) report Sharpe > 2.0 and 30-45% annual returns with 5% max drawdown. It works in production — it just hasn't been the flashy hackathon winner because hackathons reward big swings and most allow leverage.

Why It Matters for US Specifically

We have **67 coins** on Roostoo and **$50K** (not $1M). With $50K:

- A BTC-only bot puts all $50K into one asset. If BTC drops 10%, we lose $5K. That's 10% drawdown. Calmar destroyed.
- A multi-coin bot spreads $50K across the 3 strongest coins. If one drops 10%, we lose ~$1,700. Much more survivable.

Also: if BTC goes sideways for 5 days but SOL is pumping 15%, a BTC-only bot sits doing nothing while a multi-coin bot captures SOL's move.

The Warnings (Real Risks)

1. **Momentum reverses FAST in crypto** — after ~1 month it flips. Our lookback should be 7-30 days, NOT months
2. **Transaction costs eat the edge** — ranking and swapping coins frequently means more trades = more fees. With 0.05% per trade, we need each rotation to make >0.1%
3. **Some Roostoo coins might have zero volume** — other teams might not trade them, making prices unreliable
4. **More complexity = more bugs** — managing 3 positions across 3 different coins is 3x the execution logic
5. **We can't short** — research shows long/short momentum achieves 2x better Sharpe than long-only. We're limited to long-only.

My Recommendation: HYBRID

Don't go full multi-coin OR full BTC-only. Do both:

Primary (70% of capital): BTC with our 7-layer pipeline. This is our tested, understood strategy.

Secondary (30% of capital): Top 2 alt-coins by momentum ranking. Simple ranking: which coins had the highest return in the last 3 days AND have good volume. Only rotate weekly, not every cycle (saves fees). This gives us the drawdown protection of diversification while keeping BTC as the core. If BTC is ranging but ETH and SOL are running, we capture some of that move.

BUT — only implement this if we have time after the core BTC bot is working. BTC-only with a solid 7-layer pipeline beats a buggy multi-coin bot every time.